SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1707**

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Reverse Engineering

Code of Conduct:

I P.Kalyani(192472199) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Kalyani

Experiment 1: Machine Learning Techniques for Reverse Engineering

Objective: Apply machine learning techniques to source-code samples to classify software components, identify programming patterns, detect similarities/anomalies, and compare ML algorithms.

Answer: Machine Learning can support reverse engineering by converting source-code characteristics into measurable features and learning patterns from known software samples. A dataset may contain source-code files together with labels such as programming language, software component type, or functional category. Useful features include token frequencies, keyword counts, line counts, comment density, operator counts, API-call frequencies, and other static characteristics.

When labelled source-code samples are available, supervised algorithms such as Decision Tree, Random Forest, Support Vector Machine, or Logistic Regression can be used for classification. When labels are unavailable, clustering algorithms such as KMeans can group programs according to similarity. The performance of different approaches can then be compared using appropriate evaluation metrics.

Illustrative Python implementation:

```
import pandas as pd
from sklearn.model_selection import train_test_split from sklearn.feature_extraction.text import TfidfVectorizer from sklearn.ensemble import RandomForestClassifier from sklearn.metrics import accuracy_score, classification_report

data = pd.read_csv("source_code_dataset.csv")
```

```
X_text = data["code"] y
= data["label"]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(X_text)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
model = RandomForestClassifier(n_estimators=100,
random_state=42) model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

The source-code samples are transformed into numerical TF-IDF representations. The Random Forest classifier learns relationships between code tokens and their labels. Accuracy, precision, recall, and F1-score can be used to compare performance. Exact numerical results depend on the dataset and must be obtained from actual execution.

Expected Outcome:

Students evaluate the suitability of different machine-learning algorithms for software reverse-engineering tasks and understand how source-code features can be used to identify similarities and classify unknown programs.

Experiment 2: AI for Source Code Analysis and Program Understanding

Objective: Analyze an unfamiliar source-code repository using AI-assisted techniques and identify functions, classes, variables, dependencies, programming patterns, and natural-language explanations.

Answer: AI can assist source-code understanding by processing large repositories and extracting structural and semantic information. Static analysis can identify functions, classes, variables, imports, API calls, and dependencies. Natural Language Processing and Large Language Models can then transform this information into human-readable explanations.

AI-generated explanations must be verified against the actual source code. This is important because an AI system may infer an incorrect purpose, overlook an edge case, or describe functionality that is not implemented.

Illustrative Python implementation:

```
import ast
 with open("sample.py", "r", encoding="utf-8") as
f:
    source = f.read()

tree = ast.parse(source)
```

```
print("Functions:") for node in
ast.walk(tree):     if isinstance(node,
ast.FunctionDef):
        print(" -", node.name)

print("\nClasses:") for node in
ast.walk(tree):     if
isinstance(node, ast.ClassDef):
print(" -", node.name)


print("\nImported modules:") for node in
ast.walk(tree):     if isinstance(node,
ast.Import):         for item in
node.names:             print(" -",
item.name)     elif isinstance(node,
ast.ImportFrom):
        print(" -", node.module)
```

The Abstract Syntax Tree provides a structured representation of the program. Functions, classes, and imports can therefore be identified without executing the source code. AI can use these structures together with source text to generate explanations, while verification against the original implementation validates the generated result.

Expected Outcome:

Students assess the effectiveness and limitations of AI in understanding unfamiliar source code and learn that AI-generated explanations require verification against the original implementation.

Experiment 3: AI-Based Malware Reverse Engineering and Analysis

Objective: Study how AI and machine learning can assist malware analysis using safe, non-executable datasets or publicly available research samples.

Answer: AI-assisted malware analysis can reduce the manual effort required to examine large numbers of samples. A safe workflow begins with non-executable feature datasets rather than executing unknown malware. Static features may include file size, entropy, imported API names, string characteristics, section information, byte-frequency statistics, and other metadata.

After feature extraction, a supervised machine-learning classifier can be trained using labelled benign and malicious samples. The model can then classify unseen feature records. Accuracy alone is insufficient for security classification because false negatives can be important, so precision, recall, F1-score, and the confusion matrix should also be evaluated.

Illustrative Python implementation using a feature dataset:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = pd.read_csv("malware_features.csv")

X = data.drop("label", axis=1) y = data["label"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

```
model = RandomForestClassifier(n_estimators=100,
random_state=42) model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:") print(confusion_matrix(y_test,
y_pred)) print("Classification Report:")
print(classification_report(y_test, y_pred))
```

The classifier learns statistical relationships between static features and the sample labels. The confusion matrix identifies true positives, true negatives, false positives, and false negatives. Recall is particularly useful when the objective is to identify as many malicious samples as possible. Dataset quality and representativeness strongly affect model reliability.

Expected Outcome:

Students understand how AI can support automated malware analysis, identify relevant static features, apply machine-learning classification, and evaluate predictions safely without executing unknown malicious code.

Experiment 4: Reverse Engineering of APIs Using AI

Objective: Analyze a documented, open, or controlled test API to infer endpoints, parameters, request-response structures, data formats, and functionality, and generate verified documentation.

Answer: API reverse engineering is useful when documentation is incomplete or unavailable. A controlled API can be examined by sending valid requests and observing

the corresponding responses. The HTTP method, endpoint, parameters, status codes, headers, and response body provide evidence about the API's behavior.

AI can organize these observations and generate human-readable documentation. The generated documentation must be compared with the actual API specification or controlled test behavior. The purpose is to recover understanding of the interface, not to bypass authentication or access systems without authorization.

Illustrative Python implementation:

```
import requests

BASE_URL = "https://example.com/api"

response = requests.get(
BASE_URL + "/users",
params={"limit": 5},     timeout=10
)
print("Status code:", response.status_code) print("Response headers:") print(response.headers)

print("\nResponse body:")
print(response.text)
```

API Analysis Template:

```
Endpoint: /users
Method: GET
Observed parameters: limit
Purpose: Retrieve a collection of user records Response format: JSON (if confirmed from the actual response)
```

```
Status code: Record the value returned by the controlled
API
Authentication: Record only what is specified by the API
documentation
```

The request-response pair provides observable evidence about the API contract. An AI system can summarize the endpoint, parameters, response fields, and likely purpose. Every generated claim must be checked against actual API behavior. Differences between generated documentation and the real specification should be recorded as limitations or corrections.

Expected Outcome:

Students assess the usefulness of AI for API understanding and documentation recovery by comparing AI-generated documentation with the actual API specification and observed behavior.

Overall Conclusion

AI can significantly assist reverse engineering by automating repetitive analysis of source code, extracting useful features, classifying software artifacts, identifying suspicious patterns, and recovering API documentation. However, AI-generated conclusions are not automatically correct. Reliable reverse engineering requires validation against actual source code, datasets, API behavior, or other trustworthy evidence.

Note:

Exact numerical results, screenshots, and execution outputs depend on the dataset, repository, controlled API, and environment used. They should be obtained from actual execution rather than fabricated.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness